

2.2 — Implement Structured Error Responses for MCP Tools

Concept

When an MCP tool fails, how you report that failure determines whether the model can recover gracefully or gets stuck. MCP provides a first-class mechanism for this: the `isError` flag on tool results. Setting `isError: true` signals to the model that the tool call did not succeed — but the response is still a valid, parseable result, not an uncaught exception.

The critical insight is that an error response should carry **enough structured information for the model to decide what to do next**. A raw stack trace or a generic `"Something went wrong"` message gives the model nothing actionable. A structured error with a category, a message, and retry guidance gives the model a path forward.

Error Categories

Categorize every error at the point it is returned. Four categories cover the vast majority of MCP tool failures:

Category	Meaning	Retryable?
<code>validation</code>	The input was malformed or missing required fields	No — fix the input
<code>auth</code>	Missing or invalid credentials / permissions	No — escalate to user
<code>not_found</code>	The requested resource does not exist	No — verify the identifier
<code>rate_limit</code>	Too many requests in a short window	Yes — wait and retry

Retryable vs Non-Retryable

Non-retryable errors mean that retrying with the same inputs will produce the same failure. The model should stop, explain the problem to the user, and optionally ask for corrected input. Validation errors, auth failures, and missing resources all fall into this category.

Retryable errors mean the failure is transient and the same call may succeed if attempted again after a delay. Rate limits and transient network timeouts are retryable. When you mark an error retryable, include a `retryAfterMs` hint in the metadata so the model has a concrete wait target rather than guessing.

Structured Metadata

Beyond the category flag, include a `details` object with fields the model can read and act on:

- `field` — which input parameter caused a validation failure
- `resource` — the identifier that was not found
- `retryAfterMs` — milliseconds to wait before a retry
- `scope` — the permission scope that is missing for auth errors

Never include raw stack traces in error responses. Stack traces leak internal implementation details, are not actionable for a language model, and may expose sensitive path or dependency information to the caller.

Code Example

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { z } from "zod";

// Shared error-response builder – keeps all tools consistent
function toolError(
  category: "validation" | "auth" | "not_found" | "rate_limit",
  message: string,
  details: Record<string, unknown> = {},
  retryable = false
) {
  return {
    isError: true,
    content: [
      {
        type: "text" as const,
        text: JSON.stringify({
          error: { category, message, retryable, details },
        }),
      },
    ],
  };
}

const server = new McpServer({ name: "example", version: "1.0.0" });

server.tool(
  "get_document",
  { id: z.string().min(1) },
  async ({ id }) => {
    // Validation – caught by zod before we reach here, but shown for clarity
    if (!id.startsWith("doc_")) {
      return toolError(
        "validation",
        "Document ID must start with 'doc_'",
        { field: "id", received: id }
      );
    }

    const doc = await db.find(id);

    if (!doc) {
```

```

    return toolError(
      "not_found",
      `Document '${id}' does not exist`,
      { resource: id }
    );
  }

  if (!currentUser.canRead(doc)) {
    return toolError(
      "auth",
      "You do not have read access to this document",
      { scope: "documents:read", resource: id }
    );
  }

  return { content: [{ type: "text", text: JSON.stringify(doc) }] };
}

);

// Rate-limit example with retryAfterMs hint
server.tool(
  "send_message",
  { body: z.string() },
  async ({ body }) => {
    const limit = await rateLimiter.check();

    if (!limit.allowed) {
      return toolError(
        "rate_limit",
        "Too many requests – please wait before retrying",
        { retryAfterMs: limit.resetInMs },
        true // retryable = true
      );
    }

    const result = await messaging.send(body);
    return { content: [{ type: "text", text: JSON.stringify(result) }] };
  }
);

```

Common Mistakes

1. **Throwing unhandled exceptions instead of returning `isError`** — An uncaught throw terminates the tool call with no structured information. Always catch errors and return them as structured responses.

2. **Returning `isError: true` with an empty or vague message** — `"Error occurred"` tells the model nothing. The message must be specific enough for the model to explain the failure to a user.
3. **Including stack traces in error content** — Stack traces are for developers reading logs, not for language models reasoning about next steps. Sanitize before returning.
4. **Omitting the `retryable` field** — Without this flag, the model has no basis for deciding whether to retry. A missing flag is treated as non-retryable by convention, so rate limit errors without it will cause the model to give up prematurely.
5. **Mixing error shape with success shape** — If your success response is `{ data: ... }` and your error response is `{ message: ... }`, the model must guess which shape it received. Use a consistent envelope: always include `error` on failure, always omit it on success.

References

- [MCP Tool Result Specification](#)
- [Anthropic Error Handling Best Practices](#)
- [MCP TypeScript SDK — Server Tools](#)